APP测试报告对比管理系统 - AI编程助手提示词

角色定位

你是一位资深的Python全栈开发工程师，精通Flask Web框架、Pandas数据处理、Excel自动化以及系统架构设计。你的任务是开发一个**APP测试报告对比分析管理系统**，专门用于导入两个版本的测试报告（如build002和build003），自动对比分析差异，生成标准化对比报告，并永久留存所有历史对比记录。

一、核心需求

1.1 业务场景

痛点分析：

- 每次APP版本升级后，需要对比新旧版本的测试报告，分析封堵效果的差异
- 手动对比效率低、易出错，无法快速定位变化的测试项
- 历史对比记录分散，缺乏统一管理和追溯能力

解决目标：

- 自动化导入和解析测试报告Excel文件
- 智能识别差异（新增/删除/修改的测试项）
- 生成可视化对比报告
- 完整的历史记录管理和快速检索

1.2 功能模块

模块1：文件导入与管理

功能要求：

- 支持上传Excel格式的测试报告文件（.xlsx/.xls）
- 自动解析文件结构：
  - 识别Sheet列表
  - 自动检测表头位置（默认第1行，可配置）
  - 提取关键列信息

- 文件预览：
  - 在线显示前20行数据
  - 显示文件基本信息（文件名、上传时间、行数、列数、Sheet数量）
- 文件存储：
  - 保留原始文件
  - 记录文件元数据到数据库

关键点：

- 文件类型验证（仅允许Excel）
- 文件大小限制（建议50MB以内）
- 重复文件检测（相同文件名+内容MD5校验）

模块2：智能差异分析

功能要求：

- 灵活的主键配置：
  - 支持单列主键（如"测试用例ID"）
  - 支持多列组合主键（如"模块+测试项"）
  - 智能主键推荐（自动识别含ID、编号、名称等关键词的列）
- 对比规则设置：
  - 选择要对比的Sheet
  - 设置表头行位置
  - 设置忽略列（如"备注"、"时间戳"等非关键字段）
  - 空值处理策略
- 差异识别逻辑：
  - **新增记录**：build002中不存在，build003中存在
  - **删除记录**：build002中存在，build003中不存在
  - **修改记录**：主键匹配，但某个字段值发生变化
  - **移动记录**：（可选）位置变化但内容不变

技术要求：

- 使用Pandas进行高效数据处理
- 支持大数据量（10万行以内）的快速对比

- 数据类型自动转换（文本、数字、日期）

模块3：报告生成与导出

功能要求：
生成多Sheet的Excel对比报告，包含：

Sheet1 – 差异汇总概览

- 对比基本信息
- 统计表格：

```
文件1行数:        1500
文件2行数:        1650
新增测试项:       180
删除测试项:       30
修改测试项:       85
差异总计:         295
```

- 差异类型分布饼图
- 关键指标趋势图

Sheet2 – 差异详情列表

- 按差异类型分组（新增/删除/修改）
- 每条差异记录包含：
  - 主键值
  - 变更类型
  - 变更列名
  - 旧值/新值
  - 差异高亮标记

Sheet3 – 新增项明细

- 完整展示所有新增的测试项
- 按模块或类别排序

Sheet4 – 删除项明细

- 完整展示所有删除的测试项

Sheet5 – 修改项对比

- 并排显示旧值和新值
- 标注具体变化的字段

导出选项:

- Excel格式（详细报告）
- PDF格式（摘要报告）
- CSV格式（数据导出，方便二次分析）

模块4：历史记录管理

功能要求:

- 完整记录每次对比操作:
  - 任务ID
  - 操作人
  - 对比时间
  - 源文件信息（文件1名称+版本）
  - 目标文件信息（文件2名称+版本）
  - 对比配置参数
  - 差异统计结果
  - 生成的报告文件路径
- 历史记录展示:
  - 列表视图（支持分页）
  - 显示关键信息摘要
- 筛选与搜索:
  - 按时间范围筛选
  - 按文件名关键词搜索
  - 按操作人筛选
  - 按差异数量筛选
- 历史记录操作:
  - 查看详情（重新打开报告）
  - 下载报告
  - 重新对比（使用相同配置）

    - 删除记录

模块5：数据看板

功能要求：

- 统计概览卡片：
    - 总文件数
    - 总对比任务数
    - 今日任务数
    - 近7天任务趋势
- 可视化图表：
    - 任务时间趋势图（折线图）
    - 文件版本对比分布（柱状图）
    - 差异类型分布（饼图）
    - 最常对比的文件对（热力图）
- 快速操作入口：
    - 上传新文件
    - 创建新对比
    - 查看最近任务

模块6：用户与权限管理

功能要求：

- 用户注册与登录
- 基于角色的权限控制：
    - **普通用户**：
        - 上传文件
        - 创建对比任务
        - 查看自己的历史记录
        - 下载报告
    - **管理员**：
        - 查看所有用户记录
        - 管理系统文件

    - 查看系统统计
    - 用户管理

- 操作日志记录

二、技术架构

2.1 技术栈

```
前端层:
- HTML5 + CSS3 + JavaScript (ES6+)
- Bootstrap 5（UI框架）
- Chart.js 或 ECharts（数据可视化）
- SweetAlert2（友好提示）
- jQuery（DOM操作）

后端层:
- Python 3.8+
- Flask 3.0+（Web框架）
- Flask-SQLAlchemy（ORM）
- Flask-Login（用户认证）
- Pandas 2.0+（数据处理）
- OpenPyXL 3.0+（Excel读写）

数据存储:
- SQLite（开发环境，轻量级）
- PostgreSQL（生产环境，推荐）

文件存储:
- 本地文件系统（保存原始文件和生成的报告）

异步处理:
-（可选）Celery + Redis（大文件对比任务）
```

2.2 项目结构

```
app_test_comparison_system/
├── app/
│   ├── __init__.py              # Flask应用初始化
```

```
│   ├── models.py                # 数据库模型定义
│   ├── config.py                # 配置文件
│   ├── routes/                  # 路由模块
│   │   ├── __init__.py
│   │   ├── auth.py              # 认证路由（登录/注册）
│   │   ├── files.py             # 文件管理路由
│   │   ├── comparison.py        # 对比任务路由
│   │   ├── history.py           # 历史记录路由
│   │   └── dashboard.py         # 数据看板路由
│   ├── services/                # 业务逻辑层
│   │   ├── __init__.py
│   │   ├── file_service.py      # 文件处理服务
│   │   ├── diff_service.py      # 差异分析核心服务
│   │   ├── report_service.py    # 报告生成服务
│   │   └── key_detector.py      # 智能主键检测器
│   ├── utils/                   # 工具函数
│   │   ├── __init__.py
│   │   ├── excel_helper.py      # Excel辅助函数
│   │   └── validators.py        # 数据验证器
│   ├── templates/               # HTML模板
│   │   ├── base.html            # 基础模板
│   │   ├── login.html           # 登录页面
│   │   ├── register.html        # 注册页面
│   │   ├── dashboard.html       # 数据看板
│   │   ├── files.html           # 文件管理页面
│   │   ├── comparison.html      # 创建对比任务页面
│   │   ├── result.html          # 对比结果页面
│   │   ├── history.html         # 历史记录页面
│   │   └── report_preview.html  # 报告预览页面
│   ├── static/                  # 静态资源
│   │   ├── css/
│   │   │   └── style.css        # 自定义样式
│   │   ├── js/
│   │   │   ├── main.js          # 主逻辑
│   │   │   ├── file-upload.js   # 文件上传
│   │   │   └── charts.js        # 图表配置
│   │   └── images/
│   └── uploads/                 # 上传文件存储目录
├── reports/                     # 生成的报告存储目录
├── instance/                    # 实例文件夹
│   └── database.db              # SQLite数据库
├── logs/                        # 日志目录
├── requirements.txt            # Python依赖
├── config.py                    # 应用配置
├── run.py                       # 启动文件
├── deploy.sh                    # 部署脚本
└── README.md                    # 使用说明
```

2.3 数据库设计

表1：users（用户表）

```sql
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    email VARCHAR(100) UNIQUE,
    role VARCHAR(20) DEFAULT 'user',   -- 'admin' 或 'user'
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_login DATETIME
);
```

表2：files（文件表）

```sql
CREATE TABLE files (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    original_filename VARCHAR(255) NOT NULL,
    stored_filename VARCHAR(255) NOT NULL,
    file_path VARCHAR(500) NOT NULL,
    file_size INTEGER,
    file_type VARCHAR(20),
    upload_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    sheet_names TEXT,                -- JSON: ["Sheet1", "Sheet2"]
    row_count INTEGER,
    column_count INTEGER,
    md5_hash VARCHAR(32),             -- 文件MD5，用于去重
    preview_data TEXT,                -- JSON: 前5行数据预览
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

表3：comparison_tasks（对比任务表）

```
CREATE TABLE comparison_tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
```

```sql
    task_name VARCHAR(255),
    file1_id INTEGER NOT NULL,
    file2_id INTEGER NOT NULL,
    sheet_name VARCHAR(100),
    header_row INTEGER DEFAULT 0,
    key_columns TEXT,                -- JSON: ["测试用例ID"]
    ignore_columns TEXT,             -- JSON: ["备注", "时间"]
    status VARCHAR(20) DEFAULT 'success',   -- 'success', 'failed', 'pending'
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    completed_at DATETIME,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (file1_id) REFERENCES files(id),
    FOREIGN KEY (file2_id) REFERENCES files(id)
);
```

表4：comparison_results（对比结果表）

```sql
CREATE TABLE comparison_results (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    task_id INTEGER NOT NULL UNIQUE,
    file1_row_count INTEGER,
    file2_row_count INTEGER,
    added_count INTEGER,
    deleted_count INTEGER,
    modified_count INTEGER,
    unchanged_count INTEGER,
    diff_details TEXT,               -- JSON: 完整差异详情
    report_path VARCHAR(500),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (task_id) REFERENCES comparison_tasks(id)
);
```

表5：config_templates（配置模板表）

```
CREATE TABLE config_templates (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    template_name VARCHAR(100) NOT NULL,
    sheet_name VARCHAR(100),
    header_row INTEGER DEFAULT 0,
```

```sql
    key_columns TEXT,
    ignore_columns TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

三、核心代码实现指南

3.1 数据库模型（app/models.py）

```python
from flask_sqlalchemy import SQLAlchemy
from flask_login import UserMixin
from datetime import datetime
import json

db = SQLAlchemy()

class User(UserMixin, db.Model):
    """用户模型"""
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
    email = db.Column(db.String(100), unique=True)
    role = db.Column(db.String(20), default='user')
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    last_login = db.Column(db.DateTime)

    # 关系
    files = db.relationship('File', backref='user', lazy=True)
    tasks = db.relationship('ComparisonTask', backref='user', lazy=True)

    def set_password(self, password):
        """设置密码（使用werkzeug加密）"""
        from werkzeug.security import generate_password_hash
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        """验证密码"""
        from werkzeug.security import check_password_hash
```

```
        return check_password_hash(self.password_hash, password)

class File(db.Model):
    """文件模型"""
    __tablename__ = 'files'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'),
nullable=False)
    original_filename = db.Column(db.String(255), nullable=False)
    stored_filename = db.Column(db.String(255), nullable=False)
    file_path = db.Column(db.String(500), nullable=False)
    file_size = db.Column(db.Integer)
    file_type = db.Column(db.String(20))
    upload_time = db.Column(db.DateTime, default=datetime.utcnow)
    sheet_names = db.Column(db.Text)  # JSON
    row_count = db.Column(db.Integer)
    column_count = db.Column(db.Integer)
    md5_hash = db.Column(db.String(32))
    preview_data = db.Column(db.Text)  # JSON

class ComparisonTask(db.Model):
    """对比任务模型"""
    __tablename__ = 'comparison_tasks'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'),
nullable=False)
    task_name = db.Column(db.String(255))
    file1_id = db.Column(db.Integer, db.ForeignKey('files.id'),
nullable=False)
    file2_id = db.Column(db.Integer, db.ForeignKey('files.id'),
nullable=False)
    sheet_name = db.Column(db.String(100))
    header_row = db.Column(db.Integer, default=0)
    key_columns = db.Column(db.Text)  # JSON
    ignore_columns = db.Column(db.Text)  # JSON
    status = db.Column(db.String(20), default='success')
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    completed_at = db.Column(db.DateTime)

    # 关系
    file1 = db.relationship('File', foreign_keys=[file1_id])
    file2 = db.relationship('File', foreign_keys=[file2_id])
    result = db.relationship('ComparisonResult', backref='task', uselist=False)
```

```python
class ComparisonResult(db.Model):
    """对比结果模型"""
    __tablename__ = 'comparison_results'

    id = db.Column(db.Integer, primary_key=True)
    task_id = db.Column(db.Integer, db.ForeignKey('comparison_tasks.id'),
nullable=False, unique=True)
    file1_row_count = db.Column(db.Integer)
    file2_row_count = db.Column(db.Integer)
    added_count = db.Column(db.Integer)
    deleted_count = db.Column(db.Integer)
    modified_count = db.Column(db.Integer)
    unchanged_count = db.Column(db.Integer)
    diff_details = db.Column(db.Text)  # JSON
    report_path = db.Column(db.String(500))
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

class ConfigTemplate(db.Model):
    """配置模板模型"""
    __tablename__ = 'config_templates'

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'),
nullable=False)
    template_name = db.Column(db.String(100), nullable=False)
    sheet_name = db.Column(db.String(100))
    header_row = db.Column(db.Integer, default=0)
    key_columns = db.Column(db.Text)  # JSON
    ignore_columns = db.Column(db.Text)  # JSON
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

3.2 智能主键检测器（app/utils/key_detector.py）

```python
import pandas as pd
from typing import List, Optional, Tuple
import re

class KeyDetector:
    """智能主键检测器"""

    # 常见主键关键词
    KEYWORD_PATTERNS = [
        r'id$', r'_id$', r'编号', r'代码', r'名称', r'number$',
```

```
            r'code$', r'用例id', r'用例编号', r'test.*id', r'item.*id'
        ]

    def __init__(self, df1: pd.DataFrame, df2: pd.DataFrame):
        """
        初始化主键检测器

        Args:
            df1: 文件1的DataFrame
            df2: 文件2的DataFrame
        """
        self.df1 = df1
        self.df2 = df2
        self.common_columns = list(set(df1.columns) & set(df2.columns))

    def detect_key_columns(self, max_keys: int = 2) -> List[str]:
        """
        自动检测主键列

        Args:
            max_keys: 最多返回多少个主键列

        Returns:
            主键列名列表
        """
        if not self.common_columns:
            return []

        # 1. 基于关键词匹配
        keyword_matches = []
        for col in self.common_columns:
            col_lower = col.lower()
            for pattern in self.KEYWORD_PATTERNS:
                if re.search(pattern, col_lower):
                    keyword_matches.append(col)
                    break

        if keyword_matches:
            return keyword_matches[:max_keys]

        # 2. 基于数据特征检测
        # 选择唯一值比例高且数据类型合适的列
        candidates = []
        for col in self.common_columns:
            unique_ratio = len(self.df1[col].dropna().unique()) / max(1,
len(self.df1[col].dropna()))
```

```
            # 唯一值比例高（>0.5）且数据类型不是纯数值
            if unique_ratio > 0.5:
                dtype = str(self.df1[col].dtype)
                is_numeric = dtype in ['int64', 'float64', 'int32', 'float32']

                # 文本类型或ID类型的优先级更高
                score = unique_ratio
                if 'str' in dtype or 'object' in dtype:
                    score += 0.3
                elif is_numeric:
                    score += 0.1

                candidates.append((col, score))

        # 按得分排序
        candidates.sort(key=lambda x: x[1], reverse=True)

        return [col for col, _ in candidates[:max_keys]]

    def build_key_index(self, df: pd.DataFrame, key_columns: List[str]) -> dict:
        """
    构建主键索引

    Args:
      df: DataFrame
      key_columns: 主键列名列表

    Returns:
      {tuple(主键值): 行索引}
    """
        if not key_columns:
            return {}

        # 检查主键列是否存在
        for col in key_columns:
            if col not in df.columns:
                raise ValueError(f"主键列 '{col}' 不存在")

        # 构建索引
        key_index = {}
        for idx, row in df.iterrows():
            # 处理空值
            key_values = []
            for col in key_columns:
                val = row.get(col)
```

```python
                if pd.isna(val):
                    val = None
                else:
                    val = str(val).strip()
                key_values.append(val)

            # 转为元组作为字典键
            key_tuple = tuple(key_values)

            # 如果主键重复，保留第一个
            if key_tuple not in key_index:
                key_index[key_tuple] = idx

        return key_index

    def get_sample_keys(self, df: pd.DataFrame, key_columns: List[str], sample_size: int = 5) -> List[str]:
        """
        获取主键样本值

        Args:
            df: DataFrame
            key_columns: 主键列名列表
            sample_size: 样本数量

        Returns:
            主键样本值列表（字符串形式）
        """
        if not key_columns:
            return []

        samples = []
        for idx, row in df.head(sample_size).iterrows():
            key_parts = []
            for col in key_columns:
                val = row.get(col)
                if pd.isna(val):
                    key_parts.append('None')
                else:
                    key_parts.append(str(val))
            samples.append(','.join(key_parts))

        return samples
```

3.3 差异分析服务（app/services/diff_service.py）

```python
import pandas as pd
import numpy as np
from typing import Dict, List, Any, Tuple
from app.models import ComparisonTask, ComparisonResult, File, db
from app.utils.key_detector import KeyDetector
import json

class DiffService:
    """差异分析服务"""

    @staticmethod
    def compare_files(task_id: int) -> Dict[str, Any]:
        """
        执行文件对比

        Args:
            task_id: 任务ID

        Returns:
            对比结果字典
        """
        # 获取任务信息
        task = ComparisonTask.query.get(task_id)
        if not task:
            raise ValueError(f"任务 {task_id} 不存在")

        # 加载文件
        file1 = File.query.get(task.file1_id)
        file2 = File.query.get(task.file2_id)

        if not file1 or not file2:
            raise ValueError("文件不存在")

        # 读取数据
        df1 = DiffService._load_excel(
            file1.file_path,
            task.sheet_name,
            task.header_row
        )
        df2 = DiffService._load_excel(
            file2.file_path,
            task.sheet_name,
            task.header_row
        )
```

```
        # 解析配置参数
        key_columns = json.loads(task.key_columns) if task.key_columns
else []
        ignore_columns = json.loads(task.ignore_columns) if
task.ignore_columns else []

        # 移除忽略列
        if ignore_columns:
            df1 = df1.drop(columns=[col for col in ignore_columns if col in
df1.columns], errors='ignore')
            df2 = df2.drop(columns=[col for col in ignore_columns if col in
df2.columns], errors='ignore')

        # 智能检测主键（如果未指定）
        key_detector = KeyDetector(df1, df2)
        if not key_columns:
            key_columns = key_detector.detect_key_columns()
            if key_columns:
                # 更新任务配置
                task.key_columns = json.dumps(key_columns)
                db.session.commit()

        # 构建主键索引
        if key_columns:
            key_index1 = key_detector.build_key_index(df1, key_columns)
            key_index2 = key_detector.build_key_index(df2, key_columns)
        else:
            # 无主键，使用行号作为标识
            key_index1 = {i: i for i in range(len(df1))}
            key_index2 = {i: i for i in range(len(df2))}

        # 执行对比
        result = DiffService._perform_comparison(
            df1, df2, key_index1, key_index2, key_columns
        )

        # 保存结果
        comparison_result = ComparisonResult(
            task_id=task.id,
            file1_row_count=len(df1),
            file2_row_count=len(df2),
            added_count=result['added_count'],
            deleted_count=result['deleted_count'],
            modified_count=result['modified_count'],
            unchanged_count=result['unchanged_count'],
            diff_details=json.dumps(result['diff_details'],
```

```
ensure_ascii=False)
        )
        db.session.add(comparison_result)

        # 更新任务状态
        task.status = 'success'
        task.completed_at = pd.Timestamp.now()
        db.session.commit()

        return result

    @staticmethod
    def _load_excel(file_path: str, sheet_name: str, header_row: int) -> pd.DataFrame:
        """
    加载Excel文件

    Args:
      file_path: 文件路径
      sheet_name: Sheet名称
      header_row: 表头行号（0表示第一行）

    Returns:
      DataFrame
    """
        try:
            df = pd.read_excel(
                file_path,
                sheet_name=sheet_name,
                header=header_row
            )

            # 移除全空行
            df = df.dropna(how='all')

            # 标准化列名（去除前后空格）
            df.columns = df.columns.str.strip()

            return df
        except Exception as e:
            raise ValueError(f"读取Excel文件失败: {str(e)}")

    @staticmethod
    def _perform_comparison(
    df1: pd.DataFrame,
    df2: pd.DataFrame,
    key_index1: Dict[Tuple, int],
```

```
    key_index2: Dict[Tuple, int],
    key_columns: List[str]
  ) -> Dict[str, Any]:
        """
    执行具体对比逻辑

    Args:
      df1: 文件1数据
      df2: 文件2数据
      key_index1: 文件1的主键索引
      key_index2: 文件2的主键索引
      key_columns: 主键列名列表

    Returns:
      对比结果字典
    """
        all_keys = set(key_index1.keys()) | set(key_index2.keys())

        added_rows = []
        deleted_rows = []
        modified_rows = []
        unchanged_rows = []

        # 获取共同列（排除主键列）
        common_columns = list(set(df1.columns) & set(df2.columns))
        compare_columns = [col for col in common_columns if col not in
key_columns]

        for key in all_keys:
            in_file1 = key in key_index1
            in_file2 = key in key_index2

            if in_file1 and not in_file2:
                # 删除的行
                row_idx = key_index1[key]
                row = df1.iloc[row_idx].to_dict()
                deleted_rows.append({
                    'key': str(key),
                    'row_number': row_idx + 1,
                    'data': row
                })

            elif not in_file1 and in_file2:
                # 新增的行
                row_idx = key_index2[key]
                row = df2.iloc[row_idx].to_dict()
```

```
                added_rows.append({
                    'key': str(key),
                    'row_number': row_idx + 1,
                    'data': row
                })

            else:
                # 检查修改
                row_idx1 = key_index1[key]
                row_idx2 = key_index2[key]

                row1 = df1.iloc[row_idx1]
                row2 = df2.iloc[row_idx2]

                # 对比每一列
                changes = []
                for col in compare_columns:
                    val1 = DiffService._normalize_value(row1.get(col))
                    val2 = DiffService._normalize_value(row2.get(col))

                    if val1 != val2:
                        changes.append({
                            'column': col,
                            'old_value': str(row1.get(col, '')),
                            'new_value': str(row2.get(col, ''))
                        })

                if changes:
                    modified_rows.append({
                        'key': str(key),
                        'row_number_file1': row_idx1 + 1,
                        'row_number_file2': row_idx2 + 1,
                        'changes': changes,
                        'data_file1': row1.to_dict(),
                        'data_file2': row2.to_dict()
                    })
                else:
                    unchanged_rows.append({
                        'key': str(key)
                    })

        return {
            'added_count': len(added_rows),
            'deleted_count': len(deleted_rows),
            'modified_count': len(modified_rows),
            'unchanged_count': len(unchanged_rows),
```

```python
            'diff_details': {
                'added': added_rows,
                'deleted': deleted_rows,
                'modified': modified_rows,
                'unchanged': unchanged_rows,
                'key_columns': key_columns,
                'compare_columns': compare_columns
            }
        }

    @staticmethod
    def _normalize_value(value: Any) -> Optional[str]:
        """
        规范化值用于对比

        Args:
            value: 原始值

        Returns:
            规范化后的字符串值
        """
        if pd.isna(value) or value == '' or value is None:
            return None
        return str(value).strip()
```

3.4 报告生成服务（app/services/report_service.py）

```python
import pandas as pd
import os
from datetime import datetime
import uuid
from typing import Dict, Any
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils.dataframe import dataframe_to_rows

class ReportService:
    """报告生成服务"""

    REPORTS_FOLDER = 'reports'

    # 样式定义
    HEADER_FILL = PatternFill(start_color='4472C4', end_color='4472C4',
```

```
fill_type='solid')
    HEADER_FONT = Font(color='FFFFFF', bold=True, size=11)
    DATA_FONT = Font(size=10)

    ADDED_FILL = PatternFill(start_color='C6EFCE', end_color='C6EFCE',
fill_type='solid')  # 绿色
    DELETED_FILL = PatternFill(start_color='FFC7CE', end_color='FFC7CE',
fill_type='solid')  # 红色
    MODIFIED_FILL = PatternFill(start_color='FFEB9C', end_color='FFEB9C',
fill_type='solid')  # 黄色

  @staticmethod
    def generate_report(task_id: int, diff_details: Dict[str, Any]) -> str:
        """
    生成Excel对比报告

    Args:
      task_id: 任务ID
      diff_details: 差异详情字典

    Returns:
      报告文件路径
    """
        # 确保报告目录存在
        os.makedirs(ReportService.REPORTS_FOLDER, exist_ok=True)

        # 生成唯一文件名
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        report_filename = f"diff_report_{timestamp}_{uuid.uuid4().hex[:8]}.xlsx"
        report_path = os.path.join(ReportService.REPORTS_FOLDER,
report_filename)

        # 创建工作簿
        wb = Workbook()

        # 删除默认Sheet
        wb.remove(wb.active)

        # 生成各个Sheet
        ReportService._generate_summary_sheet(wb, diff_details)
        ReportService._generate_statistics_sheet(wb, diff_details)
        ReportService._generate_added_sheet(wb, diff_details['diff_details']
['added'])
        ReportService._generate_deleted_sheet(wb,
diff_details['diff_details']['deleted'])
        ReportService._generate_modified_sheet(wb,
```

```
diff_details['diff_details']['modified'])

        # 保存文件
        wb.save(report_path)

        return report_path

    @staticmethod
    def _generate_summary_sheet(wb: Workbook, diff_details: Dict[str, Any]):
        """生成差异汇总Sheet"""
        ws = wb.create_sheet('差异汇总概览')

        # 写入标题
        ws['A1'] = 'APP测试报告对比分析报告'
        ws['A1'].font = Font(size=16, bold=True)
        ws.merge_cells('A1:E1')

        # 写入生成时间
        ws['A3'] = f'生成时间: {datetime.now().strftime("%Y-%m-%d %H:%M:%S")}'

        # 写入差异统计
        row = 5
        ws[f'A{row}'] = '统计项目'
        ws[f'B{row}'] = '数值'
        ReportService._apply_header_style(ws, row)

        stats = [
            ['文件1总行数', diff_details['file1_row_count']],
            ['文件2总行数', diff_details['file2_row_count']],
            ['新增测试项', diff_details['added_count']],
            ['删除测试项', diff_details['deleted_count']],
            ['修改测试项', diff_details['modified_count']],
            ['未改变测试项', diff_details['unchanged_count']],
            ['差异总计', diff_details['added_count'] +
diff_details['deleted_count'] + diff_details['modified_count']]
        ]

        row += 1
        for key, value in stats:
            ws[f'A{row}'] = key
            ws[f'B{row}'] = value
            row += 1

        # 自动调整列宽
        ws.column_dimensions['A'].width = 20
        ws.column_dimensions['B'].width = 15
```

```python
    @staticmethod
    def _generate_statistics_sheet(wb: Workbook, diff_details: Dict[str, Any]):
        """生成统计汇总Sheet"""
        ws = wb.create_sheet('详细统计')

        # 差异类型分布
        row = 1
        ws[f'A{row}'] = '差异类型'
        ws[f'B{row}'] = '数量'
        ws[f'C{row}'] = '占比'
        ReportService._apply_header_style(ws, row)

        total = diff_details['added_count'] + diff_details['deleted_count'] +
diff_details['modified_count']

        types = [
            ('新增项', diff_details['added_count'],
ReportService.ADDED_FILL),
            ('删除项', diff_details['deleted_count'],
ReportService.DELETED_FILL),
            ('修改项', diff_details['modified_count'],
ReportService.MODIFIED_FILL),
            ('未改变', diff_details['unchanged_count'], None)
        ]

        row += 1
        for name, count, fill in types:
            ws[f'A{row}'] = name
            ws[f'B{row}'] = count
            ws[f'C{row}'] = f'{count/total*100:.2f}%' if total > 0 else '0%'

            if fill:
                ws[f'A{row}'].fill = fill
                ws[f'B{row}'].fill = fill

            row += 1

        ws.column_dimensions['A'].width = 15
        ws.column_dimensions['B'].width = 12
        ws.column_dimensions['C'].width = 12

    @staticmethod
    def _generate_added_sheet(wb: Workbook, added_rows: list):
        """生成新增项详情Sheet"""
        ws = wb.create_sheet('新增项明细')
```

```
        if not added_rows:
            ws['A1'] = '无新增项'
            return

        # 写入表头
        headers = ['主键', '行号'] + list(added_rows[0]['data'].keys())
        for col, header in enumerate(headers, start=1):
            cell = ws.cell(row=1, column=col, value=header)
            cell.fill = ReportService.HEADER_FILL
            cell.font = ReportService.HEADER_FONT

        # 写入数据
        for row_idx, row_data in enumerate(added_rows, start=2):
            ws.cell(row=row_idx, column=1, value=row_data['key'])
            ws.cell(row=row_idx, column=2, value=row_data['row_number'])

            for col_idx, (key, value) in enumerate(row_data['data'].items(),
start=3):
                ws.cell(row=row_idx, column=col_idx, value=value)
                ws.cell(row=row_idx, column=col_idx).fill =
ReportService.ADDED_FILL

        # 自动调整列宽
        for col in range(1, len(headers) + 1):
            ws.column_dimensions[chr(64 + col)].width = 15

    @staticmethod
    def _generate_deleted_sheet(wb: Workbook, deleted_rows: list):
        """生成删除项详情Sheet"""
        ws = wb.create_sheet('删除项明细')

        if not deleted_rows:
            ws['A1'] = '无删除项'
            return

        # 写入表头
        headers = ['主键', '行号'] + list(deleted_rows[0]['data'].keys())
        for col, header in enumerate(headers, start=1):
            cell = ws.cell(row=1, column=col, value=header)
            cell.fill = ReportService.HEADER_FILL
            cell.font = ReportService.HEADER_FONT

        # 写入数据
        for row_idx, row_data in enumerate(deleted_rows, start=2):
            ws.cell(row=row_idx, column=1, value=row_data['key'])
```

```
            ws.cell(row=row_idx, column=2, value=row_data['row_number'])

            for col_idx, (key, value) in enumerate(row_data['data'].items(),
start=3):
                ws.cell(row=row_idx, column=col_idx, value=value)
                ws.cell(row=row_idx, column=col_idx).fill =
ReportService.DELETED_FILL

        # 自动调整列宽
        for col in range(1, len(headers) + 1):
            ws.column_dimensions[chr(64 + col)].width = 15

    @staticmethod
    def _generate_modified_sheet(wb: Workbook, modified_rows: list):
        """生成修改项详情Sheet"""
        ws = wb.create_sheet('修改项对比')

        if not modified_rows:
            ws['A1'] = '无修改项'
            return

        # 写入表头
        headers = ['主键', '行号(文件1)', '行号(文件2)', '变化列数', '变化详情']
        for col, header in enumerate(headers, start=1):
            cell = ws.cell(row=1, column=col, value=header)
            cell.fill = ReportService.HEADER_FILL
            cell.font = ReportService.HEADER_FONT

        # 写入数据
        for row_idx, row_data in enumerate(modified_rows, start=2):
            ws.cell(row=row_idx, column=1, value=row_data['key'])
            ws.cell(row=row_idx, column=2,
value=row_data['row_number_file1'])
            ws.cell(row=row_idx, column=3,
value=row_data['row_number_file2'])
            ws.cell(row=row_idx, column=4, value=len(row_data['changes']))

            # 生成变化详情字符串
            changes_text = ';'.join([
                f"{change['column']}: {change['old_value']} -> {change['new_value']}"
                for change in row_data['changes']
            ])
            ws.cell(row=row_idx, column=5, value=changes_text)

            # 高亮标记
            for col in range(1, 6):
```

```python
                ws.cell(row=row_idx, column=col).fill = ReportService.MODIFIED_FILL

        ws.column_dimensions['A'].width = 20
        ws.column_dimensions['B'].width = 12
        ws.column_dimensions['C'].width = 12
        ws.column_dimensions['D'].width = 12
        ws.column_dimensions['E'].width = 80

    @staticmethod
    def _apply_header_style(ws, row: int):
        """应用表头样式"""
        for col in range(1, 6):
            cell = ws.cell(row=row, column=col)
            cell.fill = ReportService.HEADER_FILL
            cell.font = ReportService.HEADER_FONT
            cell.alignment = Alignment(horizontal='center')
```

四、前端页面设计要点

4.1 首页/数据看板（dashboard.html）

功能要求:

- 顶部导航栏（用户信息、退出登录）
- 统计卡片区域（4个关键指标）
- 图表展示区域（任务趋势、差异分布）
- 最近操作列表（最近5个对比任务）
- 快捷操作按钮

关键代码片段:

```html
<!-- 统计卡片 -->
<div class="row">
    <div class="col-md-3">
        <div class="stat-card">
            <h3>{{ total_files }}</h3>
            <p>总文件数</p>
        </div>
```

```html
        </div>
        <div class="col-md-3">
            <div class="stat-card">
                <h3>{{ total_tasks }}</h3>
                <p>总对比任务</p>
            </div>
        </div>
        <div class="col-md-3">
            <div class="stat-card">
                <h3>{{ today_tasks }}</h3>
                <p>今日任务数</p>
            </div>
        </div>
        <div class="col-md-3">
            <div class="stat-card">
                <h3>{{ my_tasks }}</h3>
                <p>我的任务数</p>
            </div>
        </div>
    </div>

    <!-- 图表区域 -->
    <div class="chart-container">
        <canvas id="trendChart"></canvas>
    </div>

    <script>
    // Chart.js 图表初始化
    const ctx = document.getElementById('trendChart').getContext('2d');
    new Chart(ctx, {
      type: 'line',
      data: {
        labels: {{ trend_labels|tojson }},
        datasets: [{
          label: '任务趋势',
          data: {{ trend_data|tojson }},
          borderColor: 'rgb(75, 192, 192)',
          tension: 0.1
        }]
      }
    });
    </script>
```

4.2 文件管理页面（files.html）

功能要求：

- 文件上传区域（支持拖拽上传）
- 文件列表展示（表格形式）
- 文件操作按钮（预览、下载、删除、对比）
- 筛选和搜索功能

关键功能：

```javascript
// 文件上传
$('#fileUpload').on('change', function(e) {
    const formData = new FormData();
    formData.append('file', e.target.files[0]);

    $.ajax({
        url: '/api/files/upload',
        type: 'POST',
        data: formData,
        processData: false,
        contentType: false,
        success: function(response) {
            Swal.fire('成功', '文件上传成功', 'success');
            location.reload();
        },
        error: function() {
            Swal.fire('错误', '文件上传失败', 'error');
        }
    });
});

// 文件预览
function previewFile(fileId) {
    window.open('/files/preview/' + fileId, '_blank');
}
```

4.3 对比任务创建页面（comparison.html）

功能要求：

- 文件选择下拉框（文件1和文件2）
- Sheet名称选择

- 主键列选择（支持多选）
- 忽略列选择（支持多选）
- 配置模板保存/加载
- 提交对比按钮

核心逻辑:

```
// 动态加载Sheet列表
$('#file1').change(function() {
    const fileId = $(this).val();
    $.get('/api/files/' + fileId + '/sheets', function(data) {
        $('#sheetName').empty();
        data.sheets.forEach(sheet => {
            $('#sheetName').append(`<option value="${sheet}">${sheet}</option>`);
        });
        loadColumns();
    });
});

// 动态加载列列表
function loadColumns() {
    const fileId = $('#file1').val();
    const sheetName = $('#sheetName').val();
    const headerRow = $('#headerRow').val();

    $.post('/api/files/preview', {
        file_id: fileId,
        sheet_name: sheetName,
        header_row: headerRow
    }, function(data) {
        const columns = Object.keys(data.data[0]);

        $('#keyColumns').empty();
        $('#ignoreColumns').empty();

        columns.forEach(col => {
            $('#keyColumns').append(`<option value="${col}">${col}</option>`);
            $('#ignoreColumns').append(`<option value="${col}">${col}</option>`);
        });
    });
}

// 提交对比任务
```

```javascript
$('#comparisonForm').submit(function(e) {
    e.preventDefault();

    const formData = {
        file1_id: $('#file1').val(),
        file2_id: $('#file2').val(),
        task_name: $('#taskName').val(),
        sheet_name: $('#sheetName').val(),
        header_row: parseInt($('#headerRow').val()),
        key_columns: $('#keyColumns').val() || [],
        ignore_columns: $('#ignoreColumns').val() || []
    };

    $.ajax({
        url: '/comparison/create',
        type: 'POST',
        contentType: 'application/json',
        data: JSON.stringify(formData),
        success: function(response) {
            if (response.success) {
                window.location.href = '/comparison/result/' + response.task_id;
            }
        }
    });
});
```

五、使用流程示例

场景：对比 build002 和 build003 测试报告

步骤1：登录系统

- 访问系统首页
- 使用用户名和密码登录

步骤2：上传测试报告文件

- 进入"文件管理"页面
- 点击"上传文件"按钮
- 选择并上传 `APP精细化封堵专项测试报告build002.xlsx`
- 选择并上传 `APP精细化封堵专项测试报告build003.xlsx`

步骤3：创建对比任务

- 进入"创建对比"页面
- 从下拉框选择文件1：build002
- 从下拉框选择文件2：build003
- 系统自动识别Sheet和列信息
- 设置主键列（如"测试用例ID"）
- 选择需要忽略的列（如"备注"、"时间"）
- 点击"开始对比"

步骤4：查看对比结果

- 系统自动执行对比分析
- 显示差异统计概览：
  - 新增测试项：180项
  - 删除测试项：30项
  - 修改测试项：85项
- 查看详细差异列表（可按类型筛选）
- 在线预览差异详情

步骤5：下载对比报告

- 点击"下载报告"按钮
- 生成Excel格式报告（包含5个Sheet）
- 下载报告到本地

步骤6：查看历史记录

- 进入"历史记录"页面
- 查看所有历史对比任务
- 可按时间、文件名筛选
- 可重新下载历史报告或重新对比

六、部署与扩展

6.1 本地部署

```bash
# 安装依赖
pip install -r requirements.txt

# 初始化数据库
python run.py init-db

# 创建管理员账户
python run.py create-admin

# 启动服务
python run.py run
```

6.2 生产环境部署

```bash
# 使用Gunicorn部署
gunicorn -w 4 -b 0.0.0.0:8000 "app:create_app()"

# 使用Nginx反向代理
# 配置文件示例在deploy/nginx.conf
```

6.3 扩展功能建议

1. 高级对比功能

- 支持多版本同时对比（build002 vs build003 vs build004）
- 支持文件夹批量对比
- 支持自定义差异规则（如数值型数据允许误差范围）

2. 数据可视化增强

- 差异热力图
- 版本演进趋势图
- 测试覆盖率分析

3. 协作功能

- 报告共享和评论
- 团队工作区

- 任务分配和跟踪

4. API接口

- 完整的RESTful API
- 支持第三方系统集成
- Webhook回调机制

5. 性能优化

- 大文件分块处理（>100MB）
- 异步任务队列（Celery）
- Redis缓存

七、注意事项

1. **文件安全**：
   - 上传的Excel文件仅用于对比分析，不会被修改
   - 定期清理过期的文件和报告
2. **数据备份**：
   - 定期备份数据库文件
   - 保留重要的原始文件
3. **权限控制**：
   - 严格区分用户和管理员权限
   - 用户只能查看自己的数据
4. **性能考虑**：
   - 单个Excel文件建议不超过100MB
   - 对比数据建议不超过10万行
5. **浏览器兼容性**：
   - 支持主流现代浏览器（Chrome、Firefox、Edge）
   - 建议使用Chrome以获得最佳体验

八、交付要求

请按照以上需求，开发一个功能完整、界面友好、性能稳定的APP测试报告对比分析管理系统。系统应具备：

✅ 完整的用户认证和权限管理
✅ 灵活的文件导入和解析功能
✅ 智能的差异分析引擎
✅ 多格式报告生成和导出
✅ 完整的历史记录管理和检索
✅ 友好的Web用户界面
✅ 清晰的代码结构和注释
✅ 详细的部署文档和使用说明

优先级说明：

- P0（必须实现）：文件导入、差异分析、报告生成、历史记录
- P1（重要）：数据看板、配置模板、文件预览
- P2（可选）：批量对比、API接口、PDF报告导出